
大数据与人工智能
自动化学院

Chapter03.07 优化器和学习率调整策略

3.7-1 导言

- **前情提要:** 上一节我们学会了如何使用**损失函数**这个“裁判”来评判模型预测的好坏。裁判会给出一个“损失分数”，并通过反向传播告诉我们每个参数应该朝哪个方向调整（也就是**梯度**）。
- **本章主角：优化器 (Optimizer)**
  - **它是谁？** 如果说梯度是地图上的“方向”，那优化器就是驾驶汽车的“引擎”和“司机”。它的工作就是根据梯度这个方向，去实际地更新和调整模型的每一个参数，目标是让损失分数越来越低。

本章路线图:

1. **认识引擎 (优化器):** 什么是优化器？它是如何工作的?
2. **控制油门 (学习率):** 深入了解优化器最重要的参数——**学习率**，它控制着我们学习的“步伐”大小。
3. **智能巡航 (学习率调整策略):** 学习如何让“油门”自动调整，在训练开始时“踩得深”(学习快)，在接近目标时“踩得浅”(学习精细)。

3.7-2. 优化器--寻找最低点：梯度下降的智慧

- **优化器的核心任务：** 管理并更新模型的参数，让损失越来越小。
- **核心方法：梯度下降**
  - 想象你站在一座连绵不绝的山脉上，如何下山?
  - **梯度 (Gradient):** 你脚下**最陡峭的那个上坡方向**。
  - **梯度下降 (Gradient Descent):** 你的策略很简单——只要沿着梯度的**反方向**（也就是最陡峭的下坡方向）走一小步，你就能离谷底更近一点。
  - 优化器就是通过不断重复这个“看方向、走一步”的过程，来逐步降低损失函数的值。

3.7-2.2 优化器Optimizer的基本属性和方法

下面我们学习Pytorch里面优化器的基本属性：

```
class Optimizer(object):
    def __init__(self, params, defaults):
            self.defaults = defaults
            self.state = defaultdict(dict)
            self.param_groups = []

            param_groups = [{'params':
param_groups}]
```

优化器的三大核心属性:

- `param_groups`: 这是优化器需要管理的**所有参数的列表**。
- `defaults`: 存储优化器的“默认设置”，比如**学习率 (learning rate)、动量 (momentum)** 等超参数。
- `state`: 优化器的“记忆区”。比如在使用动量时，优化器需要在这里记住前几次的更新方向。

优化器的五大核心方法:

- `zero_grad()`: **梯度清零**。因为PyTorch会累积梯度，所以在每次计算新梯度前，必须用这个方法把旧的梯度“擦干净”。
- `step()`: **执行一步更新**。当梯度计算好后，调用这个方法，优化器就会根据梯度和学习率去更新所有参数。
- `add_param_group()`: **添加新的参数组**。这在迁移学习中非常有用，可以为不同部分的网络（如预训练层和新建层）设置不同的学习率。
- `state_dict()` 和 `load_state_dict()`: **保存和加载优化器的状态**。这就像游戏的“存档”和“读档”功能，如果训练意外中断，可以从上次的状态继续，而不用从头再来。

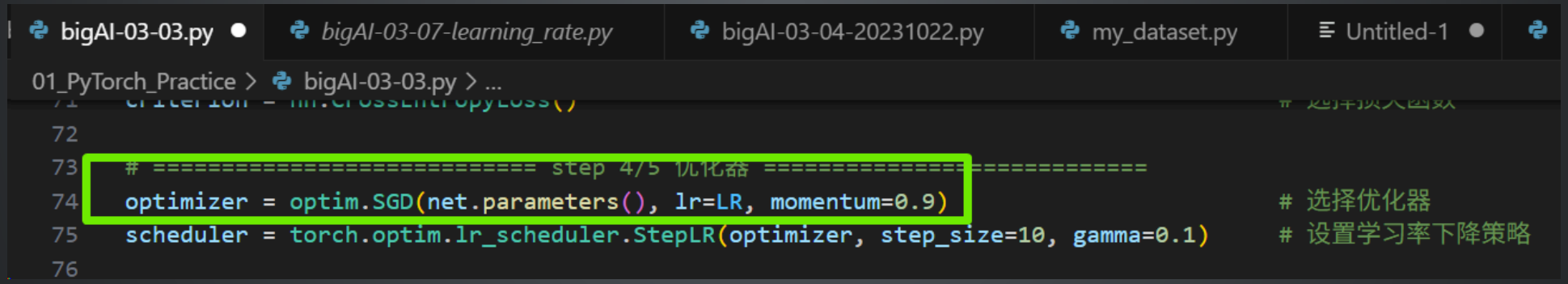
随堂作业

优化器的核心作用是根据损失函数计算出的 _____ 来更新模型的参数，目标是让损失函数的值越来越小。

答案: 梯度

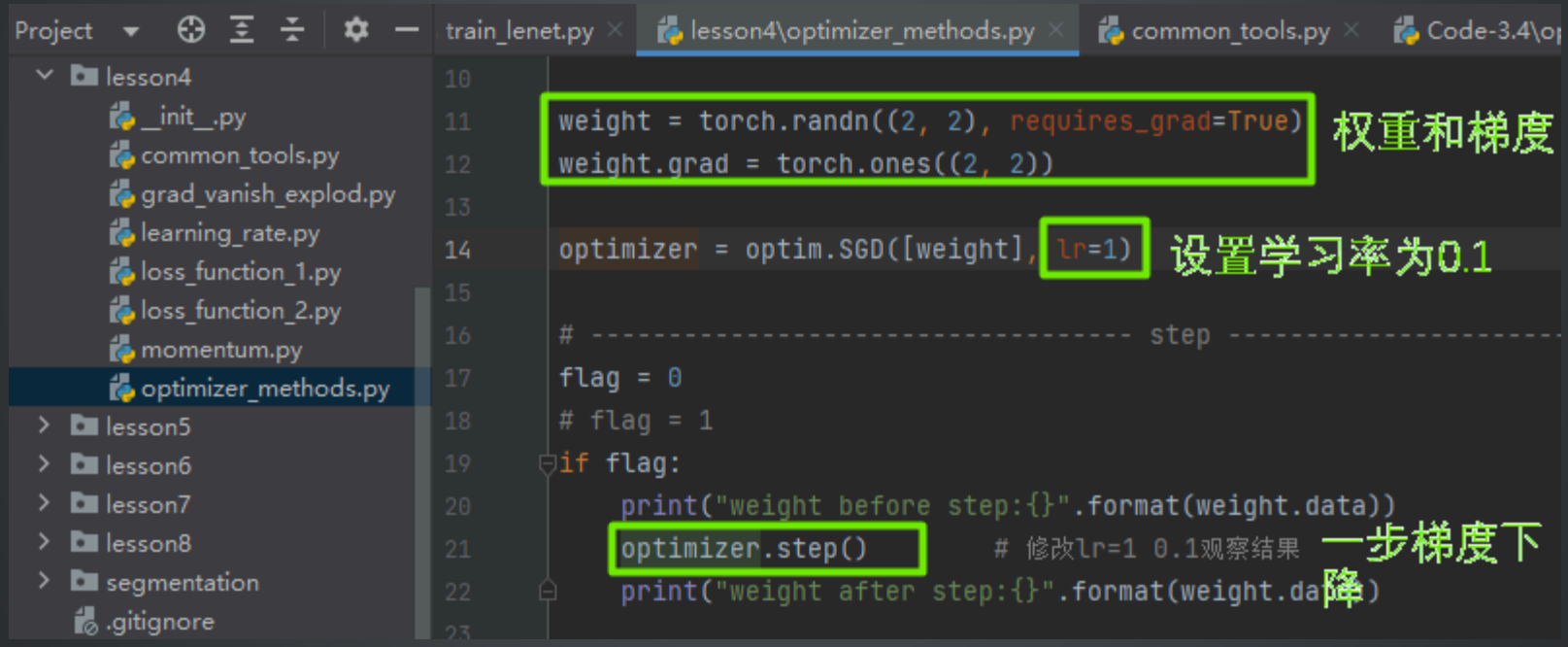
- **解析**: 整个反向传播和优化的流程是：损失函数计算损失，.backward()计算梯度，优化器根据梯度更新参数。

- 去代码中看看优化器的运行机制, 依然是代码调试的方法
- 对于人民币二分类任务的代码，进行优化器部分的调试
- 我们在优化器的定义那打上断点，然后debug



下面看优化器具体的运行：

1. step(): 一次梯度下降更新参数



2. zero_grad()： 将梯度清零

```
28      print("weight before step:{}".format(weight.data))
29      optimizer.step()        # 修改lr=1 0.1观察结果
30      print("weight after step:{}".format(weight.data))
31
32      print("weight in optimizer:{}\nweight in weight:{}\n".format(id(optimizer.param_groups[0]['params'][0]), id(weight)))
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  VARIABLES

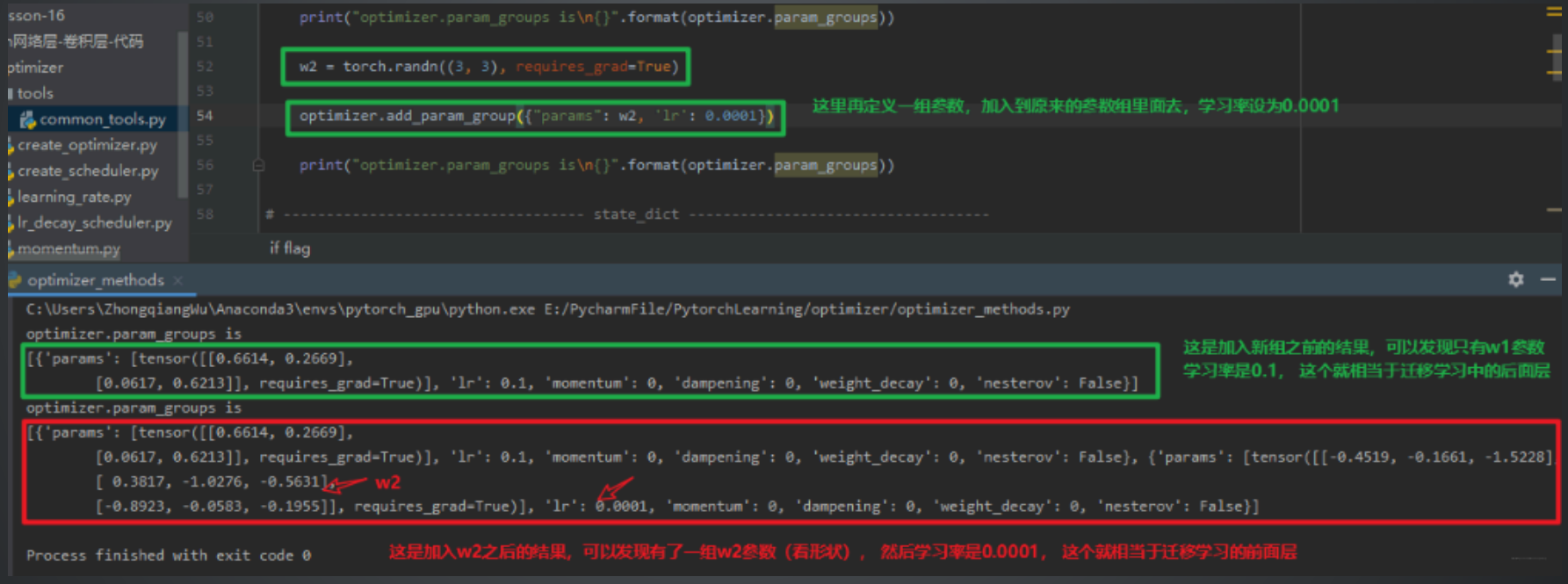
weight after step:tensor([[ 0.5614,  0.1669],
        [-0.0383,  0.5213]])
weight in optimizer:1631029474704
weight in weight:1631029474704
```

Python ..
Code
Python ..

- 使用id函数来返回两个对象的唯一标识符，这个标识符是一个整数，表示对象在内存中的地址。
- 这两个对象分别是优化器中第一个参数组的第一个参数，和weight的变量。
- 通过比较这两个对象的id值，可以判断它们是否是同一个对象
- 结果是一样的

3. add_param_group(): 添加参数组

这个是在模型的迁移学习中非常实用的一个方法，我们看看怎么用：



4. state_dict()和load_state_dict()

- state_dict()用于显示优化器信息
- load_state_dict()用于加载优化器的状态信息
- 通常用在断点的续训练， 比如我们训练一个模型，训练了10次停电了， 那么再来电的时候我们就得需要从头开始训练
- 但是如果有了这两个方法，我们就可以再训练的时候接着上次的次数继续， 所以这两个也非常实用

上面是优化器的5个方法的使用

- 了解了这些知识之后，我们就知道了优化器的运行机制，管理和更新模型的可学习参数。
  - 管理是通过各种属性，尤其是param_groups这个重要的属性
  - 更新是通过各种方法，主要是step()方法进行更新
- 那么有哪些常用的优化器呢？它们又用于什么场景呢？下面我们就来看看：

3.7-2.3 常用的优化器

- 先学习两个非常重要的概念：学习率和动量

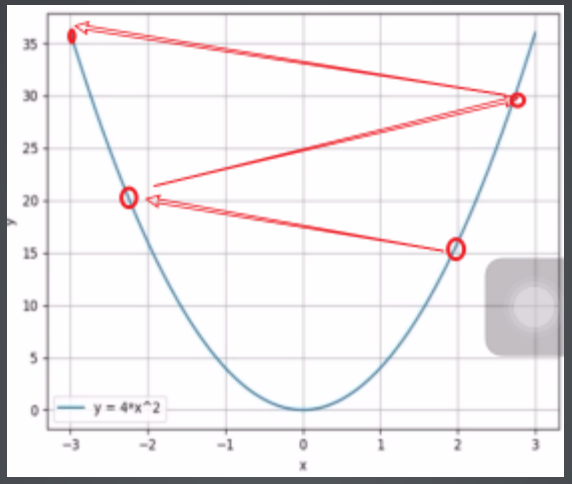
3.7-2.3.1 学习率

在梯度下降过程中，学习率起到了控制参数更新的一个步伐的作用， 参数更新公式我们都知道：

$$w_{i+1} = w_i - LR \times grad(w_i)$$

如果没有这个学习率LR的话，往往有可能由于梯度过大而错过我们的最优值

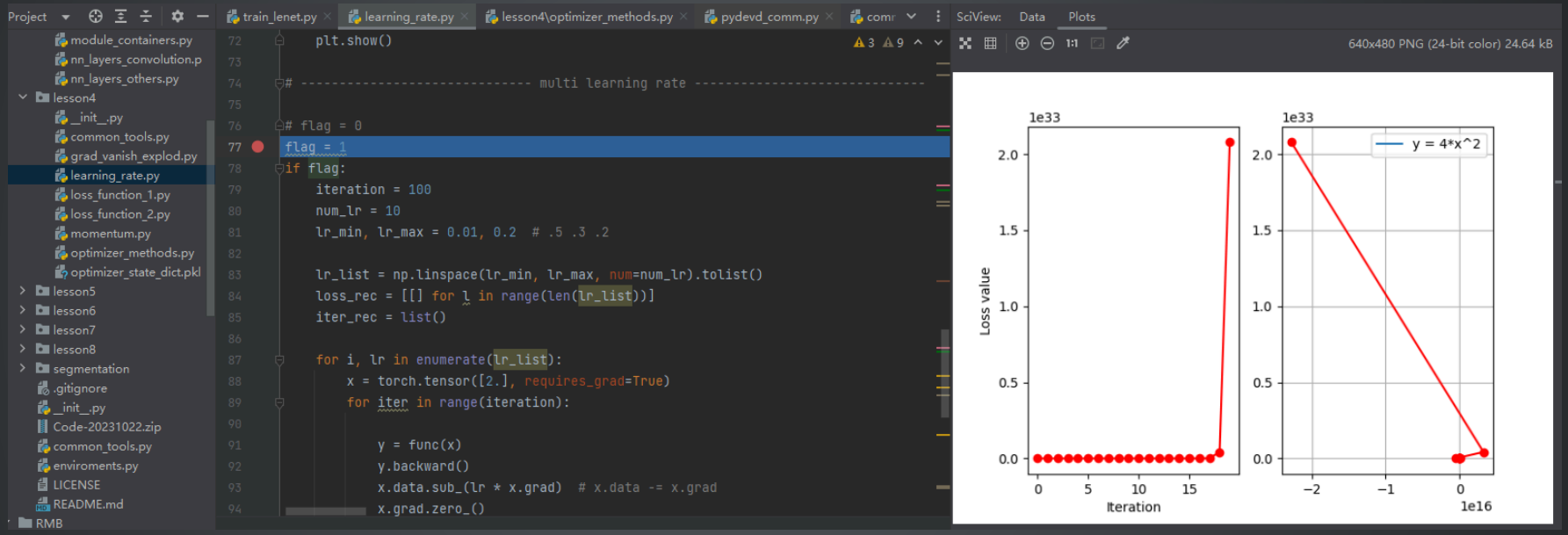
就是下面这种优化进程：



- 随着迭代次数的增加，反而越增越大， 就是因为这个步子太大了，跳过了我们的最优值。
- 这时候我们想让他这个跨度小一些，就得需要一个参数来控制我们的这个跨度，这个就是学习率。

从代码中看看：

- 学习率设置为1（很大）



我们可以看一下上面的图像，loss是不断上升的，这说明这个跨度是有问题的，

- 下面我们尝试改小一点学习率，我们就可以发现区别了：

```
iter_rec, loss_rec, x_rec = list(), list(), list()

lr = 0.01    # /1. /.5 /.2 /.1 /.125
max_iteration = 20   # /1. 4     /.5 4   /.2 20 200

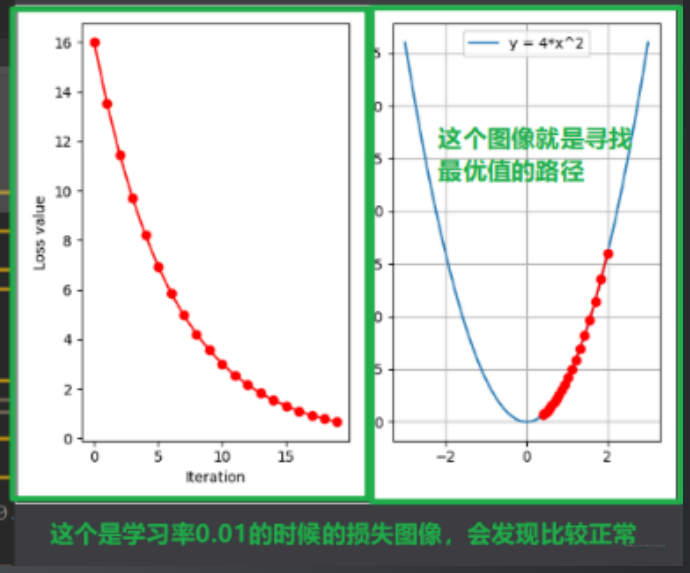
for i in range(max_iteration):

    y = func(x)
    y.backward()

    print("Iter:{}, X:{:8}, X.grad:{:8}, loss:{:10}".format(
        i, x.detach().numpy()[0], x.grad.detach().numpy()[0], y.item()))

    x_rec.append(x.item())

    x.data.sub_(lr * x.grad)    # x -= x.grad  数学表达式意义:  x = x - x.grad    # 0.
    x.grad.zero_()
```



- 如果发现：优化中的loss只升不降的时候，有可能是学习率的问题
- 所以我们一般会尝试一个小的学习率。 慢慢的去进行优化。

随堂作业

学习率（Learning Rate）控制着参数更新的“步伐”大小，如果设置得过大，可能会导致损失值在最优点附近来回 ___ ，甚至不降反升 。

答案: 震荡

- **解析**: 学习率过大会导致每一步更新的“步子”太大，可能会直接跨过损失函数的最低点，导致在谷底两侧来回跳动。

如何选择学习率?

- 学习率是一个需要仔细调整的超参数。
- 通常我们会从一个较小的值开始尝试（如0.01），或者通过实验画出不同学习率下的损失曲线，来选择一个下降最快且最稳定的值。

3.7-2.3.2 动量

动量Momentum：举个比较形象的例子：



动量的比喻： 想象一个从山上滚下来的雪球。

- **没有动量 (普通梯度下降):** 雪球只看脚下的坡度，坡有多陡就往哪走。
- **有动量:** 雪球不仅看脚下的坡度（**当前梯度**），还会保持自己之前的速度和方向（**历史梯度**）。如果连续都是下坡，它会越滚越快；即使遇到一个小坑或者平地，它也能凭借惯性冲过去。

- 动量的实现依赖于一种叫做“指数加权平均”的聪明算法。它在计算平均值时，会给**越近的数据越大的权重**，越远的数据权重越小，并且权重是按指数级衰减的。
- 在优化器中，我们用这个方法来计算梯度的“平均方向”。
- momentum系数（通常设为0.9）就像一个“记忆”参数。值越大，代表“记忆”越长，历史梯度的影响越大。

赋予更高的权重。公式如下：

$$\mathbf{v_t} = \beta \cdot \mathbf{v_{t-1}} + (1 - \beta) \cdot \theta_t$$

公式解读

1. $\mathbf{v_t}$: 当前时刻 t 的指数滑动平均值。
2. β: 平滑系数，取值范围为 $0 < \beta < 1$。它决定了历史数据的权重，值越接近 1，历史数据的影响越大；值越接近 0，最近数据的影响越大。
3. $\mathbf{v_{t-1}}$: 前一时刻 $t-1$ 的指数滑动平均值。
4. θ_t: 当前时刻 t 的原始数据点。

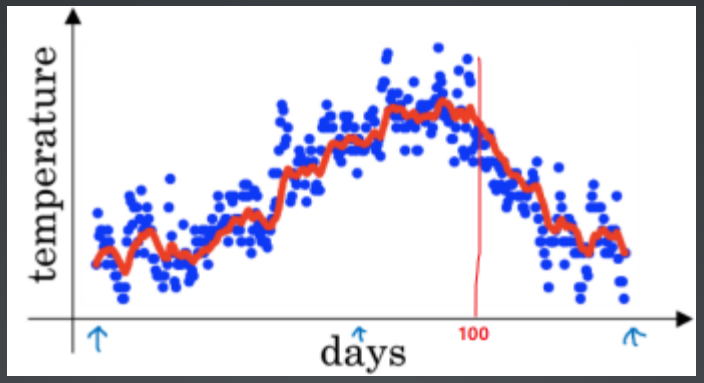
运作原理

- **递归计算**：当前的 EMA 值由前一时刻的 EMA 值和当前数据点的加权组合得到。
- **权重分配**：通过平滑系数 β，对最近的数据点（$(1 - \beta) \cdot \theta_t$）赋予更高权重，而对较早的数据（

$\beta \cdot \mathbf{v_{t-1}}$）赋予较低权重。

- **记忆性**：EMA 对所有历史数据都有记忆，但这种记忆是以指数形式衰减的，因此更关注最近的数据。

用求温度平均值的例子解释：

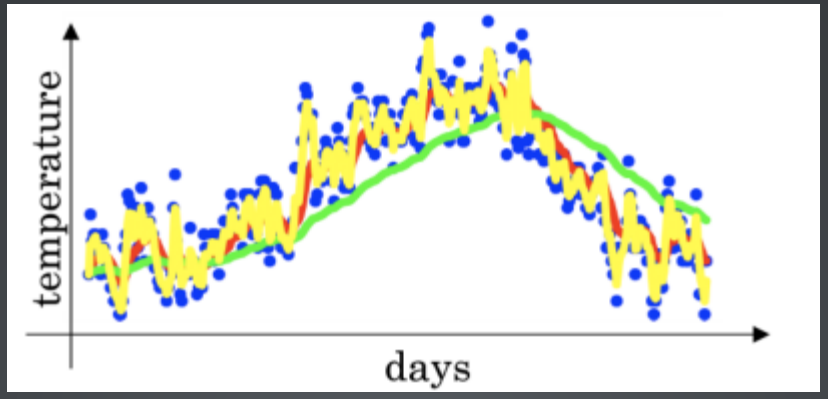


- 看上面这个温度图像，横轴是第几天，然后纵轴是温度，
- 假设我想求第100天温度的一个平均值，那么根据上面的公式：

$$
\begin{aligned}
&v_{100} = \beta * v_{99} + (1-\beta) * \theta_{100} \\
&= (1-\beta) * \theta_{100} + \beta * (\beta * v_{98} + (1-\beta) * \theta_{99}) \\
&= (1-\beta) * \theta_{100} + (1-\beta) * \beta * \theta_{99} + (\beta^2 * v_{98}) \\
&= (1-\beta) * \theta_{100} + (1-\beta) * \beta * \theta_{99} + (1-\beta) * \beta^2 * \theta_{98} + (\beta^3 * v_{97}) \\
&= (1-\beta) * \beta^0 * \theta_{100} + (1-\beta) * \beta^1 * \theta_{99} + (1-\beta) * \beta^2 * \theta_{98} + (\beta^3 * v_{97}) \\
&= \sum_i^N (1-\beta) * \beta^i * \theta_{N-i}
\end{aligned}
$$

最下面这一行就是通式

这个β参数比较重要， 还是拿求温度平均值的图片举例：



看上图，是不同beta下得到的一个温度变化曲线

- 绿色的那条，beta=0.98， 也就是考虑长时间的温度的平均值
- 红色的那条，beta=0.9， 也就是考虑中等时间的温度的平均值
- 黄色的那条，beta=0.5， 也就是考虑短时间的温度的平均

- 理解了指数滑动平均之后，就来看看Momentum
- 所谓的Momentum梯度下降， 基本的想法是**计算梯度的指数加权平均数，并利用该梯度更新权重**， 我们看看在Pytorch中是怎么实现的：
  - 普通的梯度下降：

$$w_{i+1} = w_i - lr * g(w_i)$$

  - Momentum梯度下降：

$$v_i = m * v_{i-1} + g(w_i)$$
$$w_{i+1} = w_i - lr * v_i$$

- 这里的 m就是momentum系数
- vi表示更新量，
- g(wi)是 wi的梯度。
- 这里的vi就是既考虑了当前的梯度，也考虑了上一次梯度的更新信息， 再推导一下就可以清楚：

$$
\begin{aligned}
v_{100} &= m * v_{99} + g(w_{100}) \\
&= g(w_{100}) + m * (m * v_{98} + g(w_{99})) \\
&= g(w_{100}) + m * g(w_{99}) + m^2 * v_{98} \\
&= g(w_{100}) + m * g(w_{99}) + m^2 * g(w_{98}) + m^3 * v_{97}
\end{aligned}
$$

- 可以发现，当前梯度的更新量会考虑到当前梯度， 以及前一时刻的梯度
- 这样一直往前，只不过越往前权重越小而已。

- 下面再通过代码看一下momentum的作用：
- 我们用0.01和0.03两个学习率，训练模型，我们看看loss的变化曲线：

```python
def func(x):
    return torch.pow(2*x, 2)    # y = (2x)^2 = 4*x^2        dy/dx = 8x

iteration = 100
m = 0    # .9 .63
```

此时没有加momentum

```python
lr_list = [0.01, 0.03]
```

两个不同的学习率

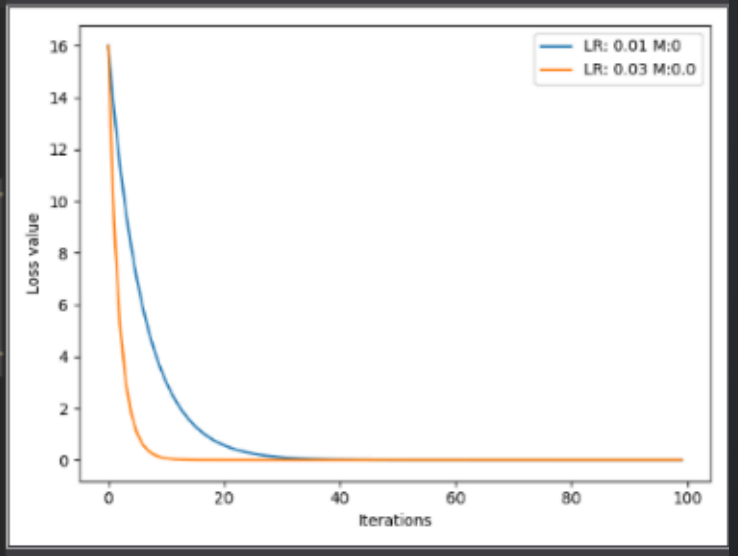
```python
momentum_list = list()
loss_rec = [[] for l in range(len(lr_list))]
iter_rec = list()

for i, lr in enumerate(lr_list):
    x = torch.tensor([2.], requires_grad=True)

    momentum = 0. if lr == 0.03 else m
    momentum_list.append(momentum)

    optimizer = optim.SGD([x], lr=lr, momentum=momentum)

    for iter in range(iteration):
```



可以发现0.03的学习率收敛速度是比较快的，毕竟步长大

现在，我们给学习率0.01的这个加一个动量momentum， 再看看效果：

```
iteration = 100
m = 0.9   # .9 .63

lr_list = [0.01, 0.03]

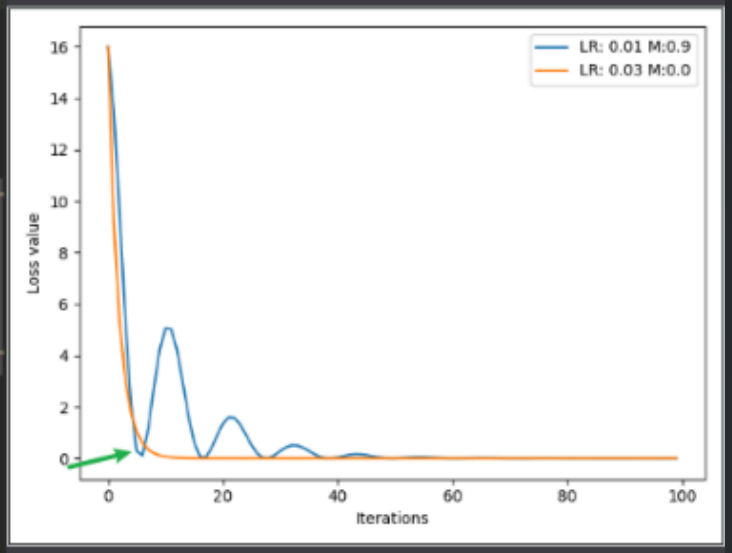
momentum_list = list()
loss_rec = [[] for l in range(len(lr_list))]
iter_rec = list()

for i, lr in enumerate(lr_list):
    x = torch.tensor([2.], requires_grad=True)

    momentum = 0. if lr == 0.03 else m
    momentum_list.append(momentum)

    optimizer = optim.SGD([x], lr=lr, momentum=momentum)

    for iter in range(iteration):
```



- 可以看到加上动量的0.01收敛的速度快了，但是前面会有震荡，
- 这是因为这里的m太大了，当日温度的权重太小，所以前面梯度一旦大小变化，这里就会震荡，
- 当然会发现震荡会越来越小最后趋于平缓，这是因为不断平均的梯度越来越多。

这时候假设我们减少动量m， 效果会好一些，比如0.63：

```
iteration = 100
m = 0.63   # .9 .63

lr_list = [0.01, 0.03]

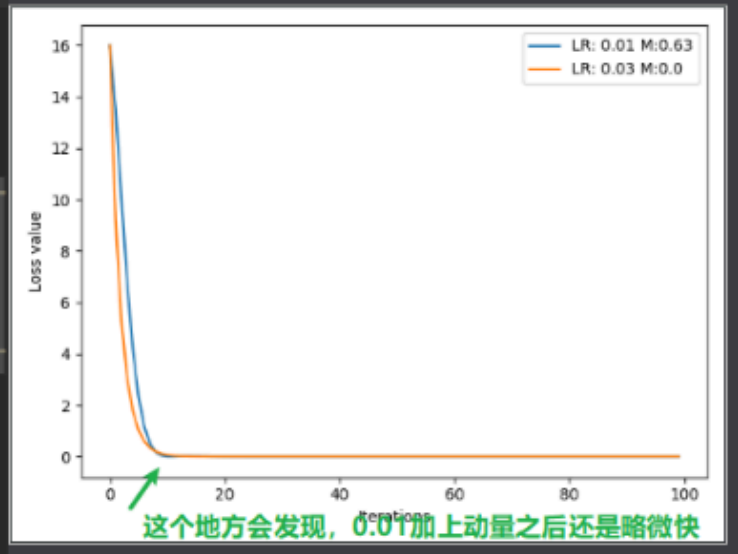
momentum_list = list()
loss_rec = [[] for l in range(len(lr_list))]
iter_rec = list()

for i, lr in enumerate(lr_list):
    x = torch.tensor([2.], requires_grad=True)

    momentum = 0. if lr == 0.03 else m
    momentum_list.append(momentum)

    optimizer = optim.SGD([x], lr=lr, momentum=momentum)

    for iter in range(iteration):
```



- 学习率和动量解释清楚了，下面就看看常用的优化器了。

随堂作业

动量（Momentum）的核心思想是引入了 _____ 梯度的影响，通过计算梯度的指数加权平均数，使得参数更新方向更加平滑和稳定。

答案: 历史 / 之前的

- **解析**: 动量就像给优化过程增加了“惯性”，它会综合考虑过去一段时间的平均梯度方向，而不是仅仅依赖当前这一个点的梯度方向。

3.7-2.3.3 常用优化器介绍

1. optim.SGD

```
optim.SGD(params, lr=<object object>,
momentum=0, dampening=0,
weight_decay=0, nesterov=False)
```

- param: 管理的参数组
- lr: 初始学习率
- momentum：动量系数，即之前的 beta
- weight_decay: L2正则化系数
- nesterov: 是否采用NAG 这个优化器很常用

然后下面列出10款优化器，具体的不去介绍， 可以大体了解有哪些优化器可用：

- optim.SGD: 随机梯度下降法
- optim.Adagrad: 自适应学习率梯度下降法
- optim.RMSprop: Adagrad的改进
- optim.Adadelta: Adagrad的改进
- optim.Adam: RMSprop结合Momentum
- optim.Adamax: Adam增加学习率上限
- optim.SparseAdam: 稀疏版的Adam
- optim.ASGD: 随机平均梯度下降
- optim.Rprop: 弹性反向传播

- 比较常用的是`optim.SGD`和`optim.Adam`
- 其他优化器的详细使用方法移步官方文档
- Ref: 优化算法的文章, 可以了解上面重要算法的一些基本原理
  https://blog.csdn.net/wuzhongqiang/article/details/1156904

3.7-3. 学习率调整策略

上面我们已经学习了优化器，在优化器当中有很多超参数，例如学习率，动量系数等，

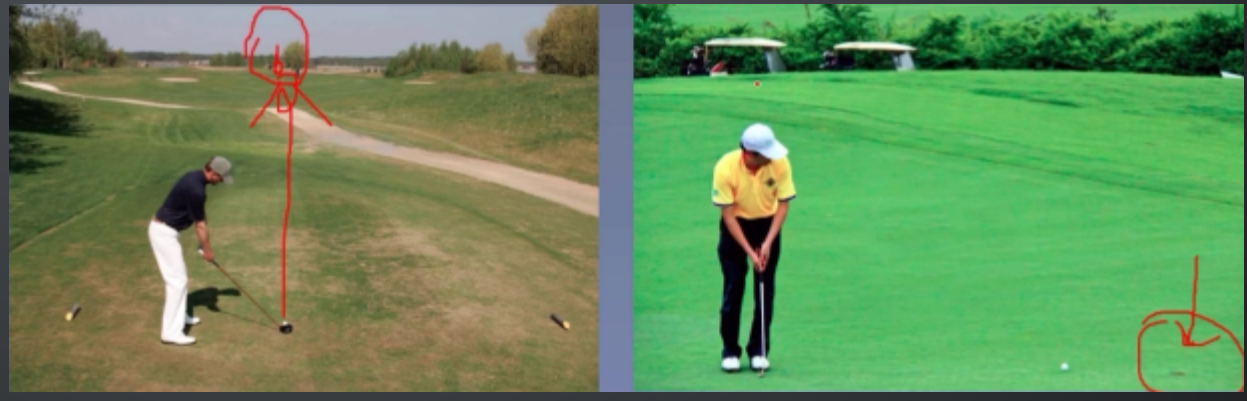
- 这里面最重要的一个参数就是学习率
  - 它直接控制了参数更新步伐的大小，整个训练当中
  - 学习率也不是一成不变的，也可以调整和变化
- 下面介绍学习率的调整策略
  - 首先是为什么要调整学习率
  - 然后是Pytorch的六种学习率调整策略
  - 最后是小结

3.7-3.1 为什么要调整学习率

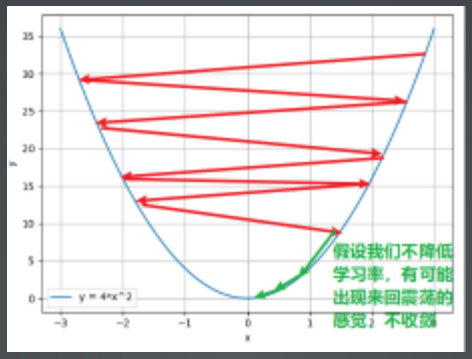
- 在整个训练过程中，一个固定的学习率通常不是最优的。**理想的策略：** 像打高尔夫球一样。

1. **训练初期 (开球):** 模型训练初期，使用一个**较大的学习率**，像大力挥杆一样，快速接近最优解区域。
2. **训练后期 (推杆入洞):** 当我们接近最优解时，需要精细调整，所以要**减小学习率**，像轻轻推杆一样，避免跨过最优点。

- 高尔夫球例子：



看一个函数的例子：



在模型的训练过程中

- **学习率前期要大，后期要小**。
- Pytorch中提供了很好的学习率的调整方法
- 下面具体学习，学习率该如何进行调整

3.7-3.2 Pytorch的学习率调度器(Scheduler)的工作原理

先学习一个基类， 因为后面的六种学习率调整策略都是继承于这个类的，所以得先明白这个类的原理：

```
class _LRScheduler(object):

    def __init__(self, optimizer, last_epoch=-1):

    def get_lr(self):
        raise NotImplementedError
```

虚函数

主要属性：

- optimizer: 关联的优化器， 得需要先关联一个优化器，然后再去改动学习率
- last_epoch: 记录epoch数， 学习率调整以epoch为周期
- base_lrs: 记录初始学习率

主要方法：

- step(): 更新下一个epoch的学习率， 这个是和用户对接
- get_lr(): 虚函数， 计算下一个epoch的学习率， 这是更新过程中的一个步骤

学习率调度器 (Scheduler) 的使用

1. **关联优化器:** 首先，我们需要创建一个调度器，并将我们的优化器“关联”给它。调度器会读取优化器中设定的初始学习率，并将其保存为`base_lrs`。
2. **调用`step()`:** 在每个epoch训练结束后，我们调用`scheduler.step()`方法。
3. **计算新学习率:** `step()`方法内部会调用一个核心的`get_lr()`函数。这个函数根据预设的规则（比如“每30个epoch学习率乘以0.1”）计算出新的学习率。

4. **更新优化器:** `step()`方法将计算出的新学习率，直接更新回关联的优化器的`param_groups`中。
5. **注意:** 调度器的`step()`通常是以epoch为单位的，要确保它放在epoch的循环中，而不是batch的循环中，否则学习率会下降得过快。

- 关于优化器的定义和使用的内部运行原理就可以稍微总结：
  - 首先我们在定义优化器的时候，这时候会完成优化器的初始化工作， 主要有关联优化器(self.optimizer属性),
  - 然后初始化last_epoch和base_lrs(记录原始的学习率，后面get_lr方法会用到)。
  - 然后就是用Scheduler，我们是直接用的step()方法进行更新下一个epoch的学习率（这个千万要注意放到epoch的for循环里面而不要放到batch的循环里面），
  - 而这个内部是在_Scheduler类的step()方法里面调用了get_lr()方法
  - 程序根据方式去计算出下一个epoch的学习率，然后直接更新进优化器的_param_groups()里面去。

下面就介绍Pytorch提供的六种学习率调整策略：

1. StepLR

```
lr_scheduler.StepLR(optimizer, step_size,
gamma=0.1, last_epoch=-1)
```

- **等间隔调整**。例如，设置`step_size=30，gamma=0.1`，表示每过30个epoch，学习率就乘以0.1

注意：

- 这里一个epoch是全部样本训练一遍
- 一个iteration，是从dataloader加载一个batch的数据的训练过程（有batch_size个样本进行训练）。也就是一个iteration就是一个batch

2. MultiStepLR

```
lr_scheduler.MultiStepLR(optimizer,
milestones, gamma=0.1, last_epoch=-1)
```

- **自定义间隔调整**。你可以提供一个列表，如 `milestones=[50, 120, 180]`，在这些特定的epoch节点上调整学习率。

3. ExponentialLR

```
lr_scheduler.ExponentialLR(optimizer, gamma,
last_epoch=-1)
```

- **指数衰减**。每个epoch都将学习率乘以一个固定的系数（如0.95），实现平滑的指数下降。

4. CosineAnnealingLR
   功能：余弦周期调整学习率

```
lr_scheduler.CosineAnnealingLR(optimizer,
T_max, eta_min=0, last_epoch=-1)
```

- **余弦退火**。学习率会像余弦曲线一样周期性地平滑下降再回升，有助于模型跳出局部最优解。

5. ReduceLROnPlateau

```
lr_scheduler.ReduceLROnPlateau(optimizer,
mode='min', factor=0.1, patience=10,
verbose=False, threshold=0.0001,
threshold_mode='rel', cooldown=0, min_lr=0,
eps=1e-08)
```

- **平台期衰减**。这是非常实用的一种策略。它会监控一个指标（如验证集损失loss或准确率acc）。
- 如果这个指标在连续好几个epoch（由patience参数设定）都没有改善，调度器就会自动降低学习率。

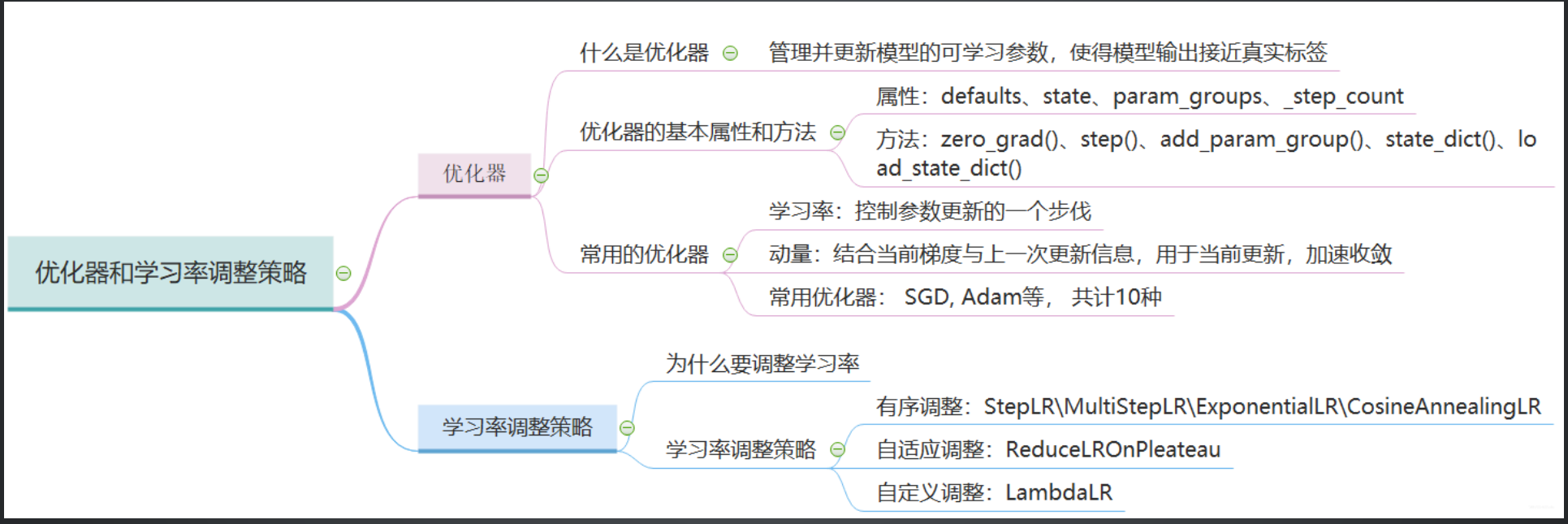
6. LambdaLR

- `lr_scheduler.LambdaLR(optimizer, lr_lambda, last_epoch=-1)`

- **自定义函数调整**。你可以自己编写一个函数来定义学习率如何随epoch变化，提供了最大的灵活性 。
- 在微调（finetune）模型时，还可以为不同的参数组设置不同的调整规则，非常强大

- **本章要点:**
  - **优化器**是模型训练的“引擎”，它使用梯度下降法来更新参数，以降低损失。
  - **学习率**和**动量**是优化器中最重要的两个超参数，分别控制更新的“步长”和“惯性”。
  - **学习率调整策略 (Scheduler)** 是实现“智能巡航”的关键，它能在训练的不同阶段自动调整学习率，以达到更优的训练效果。
- **我们的旅程:**
  - 我们已经完成了机器学习的四大核心模块：**数据 -> 模型 -> 损失函数 -> 优化器**。
  - **下一站：** 将所有模块组合起来，进行最终的**迭代训练**。

3.7-4. 总结


优化器和学习率调整策略
优化器
什么是优化器
管理并更新模型的可学习参数，使得模型输出接近真实标签
优化器的基本属性和方法
属性：defaults、state、param_groups、_step_count
方法：zero_grad()、step()、add_param_group()、state_dict()、load_state_dict()
常用的优化器
学习率：控制参数更新的一个步伐
动量：结合当前梯度与上一次更新信息，用于当前更新，加速收敛
常用优化器： SGD, Adam等， 共计10种
学习率调整策略
为什么要调整学习率
学习率调整策略
有序调整：StepLR\MultiStepLR\ExponentialLR\CosineAnnealingLR
自适应调整：ReduceLROnPleateau
自定义调整：LambdaLR